.NET frontend technologijų palyginimas

Razor Pages, Blazor Server ir Blazor WebAssembly su SimpleStudent pavyzdžiais

Modulis: Heterogeninės sistemos
Grupė: PS-5
2026 m. ruduo
Dėstytojas: Tadas Bulis
Kauno kolegija, Technologijų fakultetas

Turinys

1. Paskirtis
2. Trys frontend'ai mūsų sistemoje
3. Kaip veikia kiekviena technologija
4. Palyginimo lentelė
5. Tas pats uždavinys trimis būdais
6. CORS: kodėl jo reikia tik WebAssembly
7. Kada ką rinktis
8. Dažnos klaidos ir jų sprendimas
9. Paleidimas
10. Praktinės užduotys
11. Šaltiniai

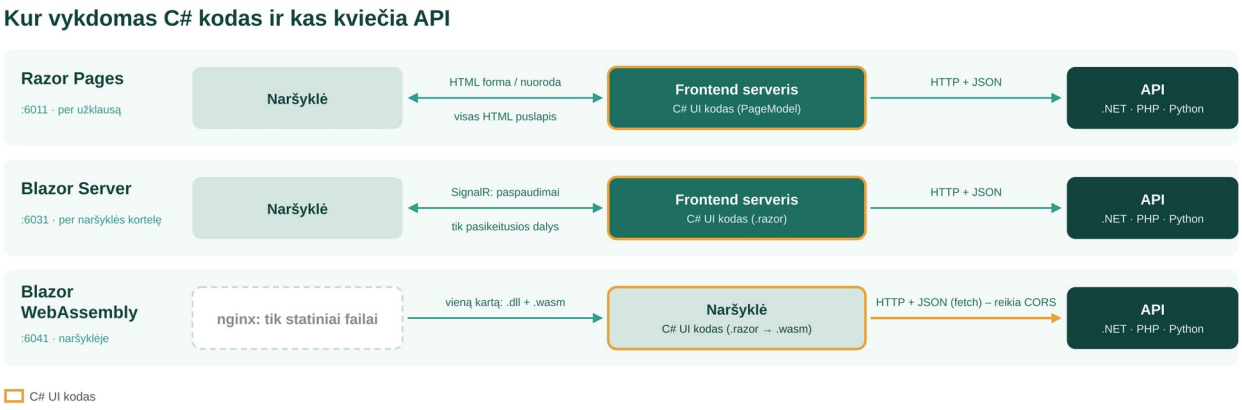
1. Paskirtis

Mūsų heterogeninėje sistemoje tas pats Students API (.NET, PHP ir Python versijos) turi kelis klientus. Trys iš jų parašyti .NET: **Razor Pages**, **Blazor Server** ir **Blazor WebAssembly**. Visi trys rodo tuos pačius duomenis ir naudoja tą patį iš `swagger.json` sugeneruotą klientą `IStudentApiClient`, tačiau veikia iš esmės skirtingai.

Šis gidas paaiškina, **kur vykdomas C# kodas**, **kas kviečia API** ir **kaip atnaujinamas puslapis** kiekvienoje technologijoje. Visi kodo pavyzdžiai paimti iš kurso projektų aplanke `student-docker-lab-plus-mssql`, todėl kiekvieną teiginį galima patikrinti paleidus projektą.

> **Pagrindinė mintis.** Razor Pages ir Blazor yra **frontend** technologijos. Nė viena jų nepakeičia API. Tai skirtingi būdai sukurti vartotojo sąsają, kuri kalbasi su tuo pačiu backend'u per HTTP ir JSON.

2. Trys frontend'ai mūsų sistemoje



1 pav. Tas pats API, trys .NET klientai. Oranžinis rėmelis rodo, kur vykdomas C# UI kodas.

Projektas	Technologija	Prievadas	Ką verta pastebėti
SimpleStudentFrontend	Razor Pages	6011	Kiekvienas veiksmas yra HTML forma arba nuoroda, po kurios naršyklė perkrauna visą puslapį.
SimpleStudentBlazorServer	Blazor Web App, Interactive Server	6031	Puslapis neperkraunamas; API perjungiklis; gyva paieška; validacija rašant.
SimpleStudentBlazorWasm	Blazor WebAssembly (standalone) + nginx	6041	Tie patys komponentai, bet vykdomi naršyklėje; F12 → Network matosi užklausos į API.

Blazor projektuose galima pasirinkti backend'ą (.NET, PHP arba Python) mygtuku viršuje dešinėje. Visi trys API įgyvendina tą patį kontraktą, todėl to paties sugeneruoto kliento pakanka visiems.

3. Kaip veikia kiekviena technologija

3.1 Razor Pages

Razor Pages yra klasikinis serverio pusės modelis: kiekvienas puslapis turi `.cshtml` šabloną ir `PageModel` klasę (`.cshtml.cs`). Kiekviena užklausa yra atskira:

1. Naršyklė siunčia `GET /Students` (arba `POST`, kai pateikiama forma).
2. Serveris sukuria `StudentsModel`, iškviečia `OnGetAsync()`, o šis per `IStudentApiClient` kviečia API.
3. Serveris sugeneruoja **visą HTML puslapį** ir išsiunčia jį naršyklei. `PageModel` objektas sunaikinamas.
4. Bet koks veiksmas (trynimas, išsaugojimas) vėl yra nauja užklausa ir naujas puslapis (dažnai `RedirectToPage()`).

3.2 Blazor Server (Interactive Server)

Blazor UI sudaro **komponentai** (`.razor` failai), kuriuose yra ir HTML, ir C# kodas (`@code { }`). Interactive Server režime komponentai vykdomi serveryje:

1. Naršyklė įkelia puslapį ir `blazor.web.js`, kuris atidaro **SignalR** (WebSocket) jungtį su serveriu.
2. Serveryje kiekvienai naršyklės kortelei sukuriama būsena, vadinama **circuit**. Komponentai ir jų laukai (`_students`, `_search`) gyvena serverio atmintyje.

3. Paspaudimas ar klavišas siunčiamas per SignalR. Serveris įvykdo C# metodą (pvz., `DeleteAsync`), palygina naują vaizdą su senu ir naršyklei išsiunčia **tik pasikeitusias DOM dalis**.
4. API kviečia serveris, kaip ir Razor Pages atveju. Naršyklė API nemato.

3.3 Blazor WebAssembly

WebAssembly režime **visa programa atsisiunčiama į naršyklę**: .NET runtime (`.wasm`), programos ir bibliotekų `.dll` failai. Tie patys `.razor` komponentai vykdomi naršyklėje:

1. nginx (arba bet kuris kitas statinių failų serveris) atiduoda `index.html`, `_framework/*.wasm` ir `*.dll`. Serveryje .NET nėra.
2. Naršyklėje paleidžiamas .NET runtime ir `Program.cs` su `WebAssemblyHostBuilder`.
3. Komponentai vykdomi naršyklėje; paspaudimams serverio nereikia.
4. API kviečia **naršyklė** per `fetch()`. Todėl API adresas turi būti pasiekiamas iš naršyklės (`localhost:6001`), o API turi leisti **CORS**.

> **.NET 8 terminologija.** Nuo .NET 8 „Blazor Web App“ šablonas vienoje programoje leidžia rinktis **render mode**: Static Server, Interactive Server, Interactive WebAssembly arba Interactive Auto (pirmą kartą Server, vėliau WebAssembly). Mūsų `SimpleStudentBlazorServer` naudoja Interactive Server visiems puslapiams, o `SimpleStudentBlazorWasm` yra atskiras (standalone) WebAssembly projektas, kad skirtumas būtų aiškiai matomas.

4. Palyginimo lentelė

Kriterijus	Razor Pages	Blazor Server	Blazor WebAssembly
Kur vykdomas UI C# kodas	Serveryje, per kiekvieną užklausą	Serveryje, per naršyklės kortelę (circuit)	Naršyklėje (WebAssembly)
Puslapio atnaujinimas	Visas puslapis po kiekvieno veiksmo	Tik pasikeitusios dalys	Tik pasikeitusios dalys
Ryšys su serveriu	Paprastos HTTP užklausos	Nuolatinė SignalR (WebSocket) jungtis	Tik failų atsisiuntimas ir API užklausos
Kas kviečia API	Frontend serveris	Frontend serveris	Naršyklė
API adresas	Vidinis: `http://simplestudentapi:8080`	Vidinis: `http://simplestudentapi:8080`	Viešas: `http://localhost:6001`
CORS API pusėje	Nereikia	Nereikia	**Reikia**
Pirmas įkėlimas	Greitas	Greitas	Lėtesnis (keli MB runtime), vėliau iš talpyklos
Interaktyvumas be JavaScript	Ribotas (formos)	Pilnas	Pilnas
Serverio apkrova	Tik užklausos metu	Atmintis kiekvienam prisijungusiam vartotojui	Minimali (statiniai failai)
Nutrūkus tinklui	Veikia kitas paspaudimas	UI sustoja, rodo „reconnect“	UI veikia, nepavyksta tik API užklausos
Slaptos reikšmės (raktai, slaptažodžiai)	Gali būti serveryje	Gali būti serveryje	**Negalima**: `wwwroot` failai ir `.dll` yra vieši
SEO / paieškos sistemos	Puikiai	Gerai su prerender	Silpnai be prerender
Talpinimas	ASP.NET Core serveris	ASP.NET Core serveris	Bet koks statinių failų serveris, CDN
Projekto SDK	`Microsoft.NET.Sdk.Web`	`Microsoft.NET.Sdk.Web`	`Microsoft.NET.Sdk.BlazorWebAssembly` + NuGet paketai
Mūsų projektas	`SimpleStudentFrontend` :6011	`SimpleStudentBlazorServer` :6031	`SimpleStudentBlazorWasm` :6041

5. Tas pats uždavinys trimis būdais

Žemiau pateikti sutrumpinti, bet tikri kodo fragmentai iš mūsų projektų. Blazor Server ir WebAssembly projektuose **puslapių kodas yra tas pats**; skiriasi tik `Program.cs`, HTML apvalkalas ir tai, kaip sukuriamas `HttpClient`.

5.1 Registracija Program.cs

Razor Pages — SimpleStudentFrontend/Program.cs

```
builder.Services.AddRazorPages();
builder.Services.AddHttpClient<IStudentApiClient, StudentApiClient>(client =>
{
    var apiBaseUrl = builder.Configuration["ApiSettings:BaseUrl"] ?? "http://localhost:6001";
    client.BaseAddress = new Uri(apiBaseUrl);
});
...
app.MapRazorPages();
```

Blazor Server — SimpleStudentBlazorServer/Program.cs

```
builder.Services.AddRazorComponents()
                .AddInteractiveServerComponents();   // komponentai vykdomi serveryje
builder.Services.AddHttpClient();
builder.Services.AddScoped<ApiConnection>();         // vienas kiekvienai kortelei (circuit)
...
app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode();
```

Blazor Server — Components/App.razor: render mode visiems puslapiams

```
<Routes @rendermode="new InteractiveServerRenderMode(prerender: false)" />
```

Blazor WebAssembly — SimpleStudentBlazorWasm/Program.cs

```
// šis kodas vykdomas NARŠYKLĖJE
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");     // <div id="app"> faile wwwroot/index.html
builder.Services.AddScoped<ApiConnection>();
await builder.Build().RunAsync();
```

5.2 Studentų sąrašo įkėlimas

Razor Pages duomenis įkelia `OnGetAsync()` metode, o šablonas juos atvaizduoja vieną kartą. Blazor komponente duomenys yra komponento laukas: pasikeitus laukui, Blazor pats perpiešia susijusią HTML dalį.

Razor Pages — Pages/Students.cshtml.cs

```
public class StudentsModel : PageModel
{
    private readonly IStudentApiClient _api;
    public StudentsModel(IStudentApiClient api) => _api = api;

    public List<Student> Students { get; private set; } = new();

    public async Task OnGetAsync()
    {
        Students = (await _api.GetStudentsAsync()).ToList();   // GET /api/students
    }
}
```

Blazor (Server ir WASM) — Pages/Students.razor

```
@page "/students"
@inject ApiConnection Api

<input class="search" @bind="_search" @bind:event="oninput" />
@foreach (var s in Filtered) { <tr><td>@s.Id</td><td>@s.FirstName</td>...</tr> }

@code {
    private List<Student> _students = new();
    private string _search = "";

    // perskaičiuojama po kiekvieno klavišo paspaudimo – API nekviečiamas
    private IEnumerable<Student> Filtered => _students.Where(s =>
        $"{s.FirstName} {s.LastName} {s.Email}"
            .Contains(_search, StringComparison.OrdinalIgnoreCase));

    protected override async Task OnInitializedAsync()
        => _students = (await Api.Client.GetStudentsAsync()).ToList();
}
```

Gyva paieška. `@bind:event="oninput"` atnaujina `_search` po kiekvieno klavišo paspaudimo, o `Filtered` filtruoja jau įkeltą sąrašą. Razor Pages tam reikėtų JavaScript arba naujos užklausos su `?search=` parametru.

5.3 Studento trynimas

Razor Pages — Pages/Students.cshtml

```
<form method="post" asp-page-handler="Delete" onsubmit="return confirm('Delete?')">
    <input type="hidden" name="id" value="@s.Id" />
    <button type="submit" class="btn btn-delete">Delete</button>
</form>
```

Razor Pages — Pages/Students.cshtml.cs

```
public async Task<IActionResult> OnPostDeleteAsync(int id)
{
    await _api.DeleteStudentAsync(id);   // DELETE /api/students/{id}
    return RedirectToPage();             // naršyklė iš naujo įkelia visą sąrašą
}
```

Blazor — Pages/Students.razor

```
@if (_confirmDeleteId == s.Id)
{
    <span>Delete?</span>
    <button @onclick="() => DeleteAsync(s.Id)">Yes</button>
    <button @onclick="() => _confirmDeleteId = null">No</button>
}
else
{
    <button @onclick="() => _confirmDeleteId = s.Id">Delete</button>
}

@code {
    private async Task DeleteAsync(int id)
    {
        await Api.Client.DeleteStudentAsync(id);
        _students.RemoveAll(s => s.Id == id);   // dingsta tik ši eilutė
    }
}
```

Razor Pages trynimas reikalauja formos, `POST` užklausos, peradresavimo ir naujo `GET`, t. y. trijų HTTP užklausų naršyklėje. Blazor atveju tai vienas C# metodas, o naršyklėje pasikeičia tik viena lentelės eilutė.

5.4 Forma ir validacija

NSwag iš `swagger.json` sugeneravo `Student` klasę su atributais `[Required]`, `[StringLength]` ir `[Range]`. Blazor `DataAnnotationsValidator` juos tikrina rašant, todėl **validacijos taisyklės ateina iš API kontrakto**. Klaidas, kurias grąžina API (400 ValidationProblem), Blazor parodo prie atitinkamų laukų.

Razor Pages — Pages/StudentCreate.cshtml(.cs)

```
<form method="post">
    <input asp-for="Input.FirstName" required />
    <input asp-for="Input.EnrollmentYear" type="number" min="2000" max="2100" required />
    <button type="submit">Save Student</button>
</form>

[BindProperty] public StudentDto Input { get; set; } = new();

public async Task<IActionResult> OnPostAsync()
{
    try { await _api.CreateStudentAsync(Input.ToApi()); return RedirectToPage("/Students"); }
    catch (Exception ex) { ErrorMessage = ApiErrors.ToMessage(ex); return Page(); }
}
```

Blazor — Shared/StudentForm.razor (viena forma kūrimui ir redagavimui)

```
<EditForm EditContext="_editContext" OnValidSubmit="SubmitAsync" novalidate>
    @* taisyklės iš sugeneruotos Student klasės *@
    <DataAnnotationsValidator />
    <InputText @bind-Value="Model.FirstName" />
    <ValidationMessage For="() => Model.FirstName" />
    ...
</EditForm>

@code {
    private async Task SubmitAsync()
    {
        try { await Save(Model); }        // Create... arba UpdateStudentAsync
        catch (Exception ex)
        {
            // 400: { "errors": { "firstName": ["First name is required."] } }
            foreach (var (field, messages) in ApiErrors.FieldErrors(ex))
            {
                var property = char.ToUpperInvariant(field[0]) + field[1..];
                _serverErrors.Add(_editContext.Field(property), messages);
            }
            _editContext.NotifyValidationStateChanged();
        }
    }
}
```

> **Spąstai: novalidate.** Laukas `type="email"` įjungia naršyklės HTML5 validaciją: su reikšme „x“ forma tiesiog nepateikiama, o klaidos nesimato nei Blazor, nei API. Atributas `novalidate` ant `EditForm` išjungia naršyklės patikrą, ir tada klaidas rodo Blazor bei API. Šią klaidą radome testuodami pavyzdį.

5.5 API adresas ir HttpClient

Svarbiausias praktinis skirtumas: **kas siunčia HTTP užklausą**. Serverio pusės frontend'ai Docker tinkle naudoja vidinį vardą, o WebAssembly naudoja adresą, kurį mato naršyklė.

Blazor Server — docker-compose.yml

```
Backends__0__BaseUrl: http://simplestudentapi:8080   # serveris → API (Docker tinkle)
Backends__0__PublicUrl: http://localhost:6001        # naršyklė → Swagger nuoroda
```

Blazor WebAssembly — wwwroot/appsettings.json (viešas failas!)

```
{ "Name": ".NET", "Url": "http://localhost:6001" }
```

Services/ApiConnection.cs abiejuose Blazor projektuose

```
// Blazor Server: HttpClient iš IHttpClientFactory serveryje
var http = _httpClientFactory.CreateClient();
http.BaseAddress = new Uri(backend.BaseUrl);

// Blazor WebAssembly: HttpClient = naršyklės fetch()
new StudentApiClient(new HttpClient { BaseAddress = new Uri(Current.Url) });
```

6. CORS: kodėl jo reikia tik WebAssembly

CORS (Cross-Origin Resource Sharing) yra naršyklės saugumo taisyklė: JavaScript (ir WebAssembly) iš vieno adreso (`http://localhost:6041`) gali skaityti atsakymą iš kito adreso (`http://localhost:6001`) tik tada, kai pastarasis tai leidžia antraštėmis `Access-Control-Allow-*`.

- Razor Pages ir Blazor Server API kviečia **serveris**. Serveriui CORS taisyklės negalioja, todėl API nieko papildomai daryti nereikia.
- Blazor WebAssembly API kviečia **naršyklė**. Prieš `POST`/`PUT` su JSON naršyklė siunčia **preflight** užklausą `OPTIONS`; jei API neatsako teisingomis antraštėmis, užklausa blokuojama.
- PHP API CORS turėjo nuo pradžių (`Http::cors()`), Python API turi `CORSMiddleware`, o .NET API buvo papildytas kodu žemiau.

.NET API — SimpleStudentApi/Program.cs

```
builder.Services.AddCors(options =>
    options.AddDefaultPolicy(policy => policy
        .AllowAnyOrigin()
        .AllowAnyHeader()
        .AllowAnyMethod()
        .WithExposedHeaders("Location")));   // naršyklė mato 201 antraštę Location
...
app.UseCors();
```

> **Gamyboje.** `AllowAnyOrigin()` tinka mokomajam projektui. Realiame viešame API reikėtų išvardinti leidžiamus adresus, pvz. `WithOrigins("https://students.example.com")`.

7. Kada ką rinktis

Rinkitės	Kai…	Atsargiai, jei…
Razor Pages	Turinio svetainė, formos, administravimo puslapiai; svarbus SEO; reikia paprasčiausio ir patikimiausio sprendimo.	Reikia daug interaktyvumo be perkrovimų – tektų rašyti JavaScript.
Blazor Server	Vidinė verslo programa; daug interaktyvumo; greitas pirmas įkėlimas; API ir DB yra privačiame tinkle; kodas turi likti serveryje.	Daug vienu metu prisijungusių vartotojų (atmintis serveryje) arba nestabilus tinklas (vėlavimas, atsijungimai).
Blazor WebAssembly	Programa turi veikti ir nutrūkus ryšiui (PWA); talpinimas kaip statiniai failai/CDN; norima perkelti skaičiavimus į klientą.	Didelis pirmas atsisiuntimas; visi `.dll` ir nustatymai vieši; API turi būti viešas, su CORS ir autentifikacija (tokenais).

Heterogeninėje sistemoje nėra vieno teisingo atsakymo: mūsų lab'e trys .NET frontend'ai, PHP frontend'as ir Flutter programa naudoja tą patį API. Sprendimą lemia vartotojai, tinklas ir talpinimas, o ne API.

8. Dažnos klaidos ir jų sprendimas

Problema	Kur pasitaiko	Priežastis ir sprendimas
API kviečiamas du kartus atidarant puslapį	Blazor Server	Prerender: puslapis pirma sugeneruojamas serveryje, po to dar kartą interaktyviai. Mūsų pavyzdyje išjungta: `InteractiveServerRenderMode(prerender: false)`.
„Attempting to reconnect to the server…“	Blazor Server	Nutrūko SignalR jungtis (perkrautas konteineris, tinklas). Puslapis prisijungs pats; circuit būsena prarandama.
Paspaudus „Save“ nieko nevyksta	Blazor	Naršyklės HTML5 validacija (`type="email"`) blokuoja formą. Pridėkite `novalidate` prie `EditForm`.
„TypeError: Failed to fetch“	Blazor WASM	API neveikia arba neleidžia CORS. F12 → Console parodo trūkstamą antraštę.
Adresas `simplestudentapi:8080` neveikia	Blazor WASM	Šis vardas egzistuoja tik Docker tinkle. Naršyklei reikia `localhost:6001`.

Problema	Kur pasitaiko	Priežastis ir sprendimas
Pakeistas `wwwroot/appsettings.json` neveikia	Blazor WASM	Naršyklė jį laiko talpykloje: Ctrl+F5 (Cmd+Shift+R).
API pasirinkimas dingsta po F5	Blazor Server	Taip ir turi būti: pasirinkimas saugomas circuit būsenoje, o perkrovus sukuriamas naujas circuit.

9. Paleidimas

Iš aplanko `student-docker-lab-plus-mssql` paleiskite SQL Server (vieną kartą) ir visus servisus:

Terminalas

```
docker-compose -f docker-compose.infra.yml up -d
docker-compose up --build
```

Adresas	Kas tai
http://localhost:6011	Razor Pages frontend'as
http://localhost:6031	Blazor Server frontend'as
http://localhost:6041	Blazor WebAssembly frontend'as
http://localhost:6021	PHP frontend'as
http://localhost:6001/swagger · :6002 · :6003	.NET · PHP · Python API su Swagger UI

Kiekvieno projekto `README.md` (EN) ir `README_LT.md` aprašo ir paleidimą be Docker (`dotnet run`).

10. Praktinės užduotys

1. Paleiskite visus tris .NET frontend'us. Kiekviename atidarykite F12 → **Network** ir ištrinkite po vieną studentą. Kiek užklausų išsiuntė naršyklė ir į kokius adresus? Paaiškinkite skirtumą.
2. Blazor Server puslapyje sustabdykite konteinerį `simplestudentblazorserver` (`docker stop simplestudentblazorserver`). Kas nutinka puslapiui? Pakartokite su `simplestudentblazorwasm`.
3. Iš .NET API laikinai pašalinkite `app.UseCors();`, perbuildinkite API ir atidarykite WebAssembly frontend'ą. Kokią klaidą rodo Console? Kodėl Razor Pages ir Blazor Server veikia toliau?
4. Blazor `Students.razor` pridėkite rūšiavimą pagal pavardę (paspaudus stulpelio antraštę) be naujos API užklausos.
5. Razor Pages frontend'e sukurkite puslapį **Faculty** (`GET /api/faculty`), analogišką Blazor `Faculty.razor`. Palyginkite, kiek kodo prireikė grupių išskleidimui.
6. Blazor perjunkite į PHP API ir bandykite išsaugoti studentą su el. paštu „x“. Iš kur atėjo klaidos pranešimas: iš naršyklės, iš Blazor ar iš API?

11. Šaltiniai

- ASP.NET Core Razor Pages: https://learn.microsoft.com/aspnet/core/razor-pages/
- ASP.NET Core Blazor: https://learn.microsoft.com/aspnet/core/blazor/
- Blazor render modes: https://learn.microsoft.com/aspnet/core/blazor/components/render-modes
- Blazor hosting models: https://learn.microsoft.com/aspnet/core/blazor/hosting-models

- CORS in ASP.NET Core: https://learn.microsoft.com/aspnet/core/security/cors
- NSwag (OpenAPI → C# client): https://github.com/RicoSuter/NSwag
- Kurso projektai: code-examples/student-docker-lab-plus-mssql (README.md / README_LT.md kiekviename projekte)